- The Language class applies all the previous pipeline steps to your input sentence in the background. After applying `nlp` to the sentence, the Doc object contains tokens that are tagged, lemmatized, and marked as entities if the token is an entity. Each pipeline component has a well-defined task :

NAME	COMPONENT	CREATES	DESCRIPTION
tokenizer	Tokenizer	Doc	Segment text into tokens.
tagger	Tagger	Doc[i].tag	Assign part-of-speech tags.
parser	DependencyParser	Doc[i].head, Doc[i].dep, Doc.sents, Doc.noun_chunks	Assign dependency labels.
ner	EntityRecognizer	Doc.ents, Doc[i].ent_iob, Doc[i].ent_type	Detect and label named entities

Figure – Pipeline components and tasks

spaCy provides helper classes for vectors, language vocabulary, and annotations. Vocab is the vocabulary of a language. It contains all the words of the language model we loaded : The Doc object abstracts the text by owning the sequence of tokens and all their properties.

The Vocab object provides a centralized set of strings and lexical attributes to all other classes. In this way, spaCy avoids storing multiple copies of linguistic data :

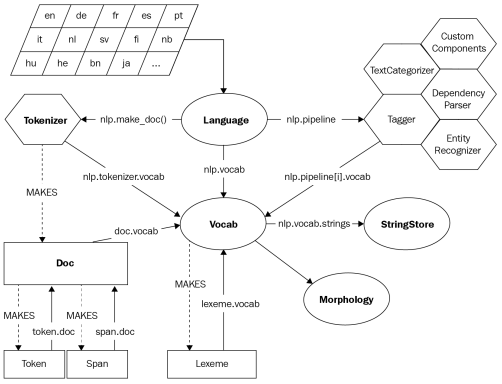


Figure – spaCy's Architecture

spaCy performs all these operations for us.

Introduction to Tokenization I

We have seen that the first step of a text processing pipeline is tokenization. Tokenization is always the first operation because all other operations require tokens.

Tokenization simply means dividing the sentence into its tokens. A token is a unit of semantics. You can think of a token as the smallest meaningful part of a piece of text. Tokens can be words, numbers, punctuation marks, currency symbols, and any other significant symbol that are the building blocks of a sentence.

The input to spaCy's tokenizer is a Unicode text and the result is a Doc object. The following code shows the tokenization process :

```
import spacy
nlp = spacy.load("en_core_web_md")
doc = nlp("put your text here.")
print ([token.text for token in doc])
['put', 'your', 'text', 'here', '.']
```

Here is what we just did :

1. We start by importing spaCy.
2. Then, we loaded the English language model via the en shortcut to create an instance of the Language class nlp.

3. Next, we apply the nlp object to the input sentence to create a Doc object, doc. A Doc object is a container for a sequence of Token objects. spaCy generates the Token objects implicitly when we created the Doc object.
4. Finally, we print a list of the tokens from the previous sentence.

There are many aspects we need to pay attention to : punctuation, white spaces, numbers, etc. It might be tempting to do a split from white spaces with text.split(" ") and it seems to work for the example sentence above.

What about the sentence "It's been a crazy week ! ! !" ? If we do a split(" "), the resulting tokens would be It's, been, a, crazy, week ! ! !, which is not what you want.

First of all, "It's" is not a token, it's two tokens : "it" and "'s". "week ! ! !" is not a valid token because the punctuation is not properly separated.

Moreover, "!!!" should be tokenized by symbol and should generate three "!". This is important for sentiment analysis. This time, the sentence is divided as follows :

0	1	2	3	4	5	6	7	8
It	‘s	been	a	crazy	week	!	!	!

Figure – Tokenization with apostrophe and punctuations

Figure 2.8 - Tokenization of the apostrophe and punctuation marks

```
1  import spacy
2  nlp = spacy.load("en_core_web_md")
3  doc = nlp("It’s been a crazy week!!!")
```

```
print ([token.text for token in doc])
['It', "'s", 'been', 'a', 'crazy', 'week', '!', '!', '!']
#Note that spaCy provides non-destructive tokenization, which means we will always be able to recover the original text from the tokens. Information about whitespace and punctuation is preserved during tokenization, so the input text is kept as is.
```

Customizing the tokenizer I

When working in a specific domain such as medicine, insurance, or finance, we often encounter words, abbreviations, and entities that require special attention. Most of the domains you will deal with have characteristic words and phrases that require custom tokenization rules. Here's how to add a special case rule to an existing Tokenizer class instance :

```
import spacy
from spacy.symbols import ORTH
nlp = spacy.load("en_core_web_md")
doc = nlp("lemme that")
print([w.text for w in doc])
['lemme', 'that']
special_case = [{ORTH: "lem"}, {ORTH: "me"}]
nlp.tokenizer.add_special_case("lemme", special_case)
print([w.text for w in nlp("lemme that")])
['lem', 'me', 'that']


```

Here's what we did :

1. We started again by importing spacy.
2. Then, we imported the ORTH symbol, which stands for orthography, i.e., text.
3. We continued creating a Language class object, nlp, and creating a Doc object, doc.
4. We defined a special case, where the word "lemme" should be tokenized into two tokens, "lem" and "me".
5. We added the rule to the tokenizer of the nlp object.
6. The last line shows how the new rule works.

Sentence segmentation I

We saw that cutting a sentence into tokens is not an easy task at all. And what about cutting a text into sentences ? It is indeed a bit more complicated to mark where a sentence starts and ends due to the same reasons as punctuation, abbreviations, etc. The sentences of a Doc object are accessible via the doc.sents property :

Therefore, spaCy uses the dependency parser to perform sentence segmentation.

This is a unique feature of spaCy. The results are usually very accurate, unless you are dealing with a text of a very specific genre, such as conversation or social media.

```
import spacy
nlp = spacy.load("en_core_web_md")
text = "I flied to N.Y yesterday. It was around 5 pm."
doc = nlp(text)
for sent in doc.sents:
print(sent.text)
I flied to N.Y yesterday.
It was around 5 pm.
```

Understanding lemmatization I

A lemma is the base form of a token. You can consider a lemma as the form under which the token appears in a dictionary. For example, the lemma of "eating" is "eat"; the lemma of "eats" is "eat"; "ate" is also mapped to "eat". Lemmatization is the process of reducing the forms of words to their lemmas. The following code is a quick example of how to do lemmatization with spaCy :

```
import spacy
nlp = spacy.load("en_core_web_md")
doc = nlp("I went there for working and worked for 3 years.")
for token in doc:
print(token.text, token.lemma_)
```

Result :

```
I -PRON-
went go
there there
for for
working work
and and
worked work
```

```
8  for for
9  3 3
10 years year
11 . .
```

In the first line, you see "-PRON-", which doesn't look like a real token. It's a pronoun lemma, a special token for the lemmas of personal pronouns. This is an exception for semantic purposes : personal pronouns "you", "I", "me", "him", "his", and so on look different, but in terms of meaning, they are part of the same group. spaCy offers this trick for pronoun lemmas.

Lemmatization in NLU I

Lemmatization is an important step in NLU. We will see an example in this subsection. Suppose you are designing a natural language processing pipeline for a ticket booking system. Your application processes a customer's sentence, extracts the necessary information, and then passes it on to the booking API.

The natural language processing pipeline needs to extract the form of travel (plane, bus, or train), the destination city, and the date. The first thing the application needs to check is the mode of transport : We have this list of keywords and we want to recognize the mode of transport by searching for tokens in the keyword list. The most compact way to

Lemmatization in NLU II

perform this search is to check the lemma of the token. Consider the following customer sentences :

Here, we don't need to include all forms of the verb "fly" (fly, flying, flies, flew, and flown) in the keyword list, and the same goes for the word "flight". We have reduced all possible variants to base forms - fly and flight. Don't just think in English ; languages like Spanish, German, and Finnish also have many word forms from a single lemma.

Lemmatization is also useful when we want to recognize the destination city because often there are many nicknames available cities. In this case, you can add special rules.

Lemmatization in NLU III

Define a special case for the word Angeltown by replacing its lemma with the official name Los Angeles.

```
# Be carreful in v3 of spaCy LEMMA is replaced with NORM
from spacy.symbols import ORTH, NORM
nlp = spacy.load('en')
special_case = [{ORTH: 'Angeltown', NORM: 'Los Angeles'}]
nlp.tokenizer.add_special_case(u'Angeltown', special_case)
doc = nlp(u'I am going to Angeltown')
for token in doc:
print(token.text, token.norm_)
I I
am be
going go
to to
Angeltown Los Angeles
```

We defined a special case for the word Angeltown by replacing its lemma with the official name Los Angeles. Then, we added this special case to the Tokenizer instance. When we display the lemmas of the tokens, we see that Angeltown is associated with Los Angeles as we wanted. It is worth noting that in NLP, a lemma is the base form of a word and is always a member of the language's vocabulary. The stem (root) does not need to be a valid word at all. For example, the lemma of "improvement" is "improvement", but the stem is "improv". You can consider the stem as the smallest part of the word that carries meaning.

spaCy Container Objects I

- The container objects in spaCy follow the natural structure of the text : a document is composed of sentences and sentences are composed of tokens.

We most often use the Doc, Token, and Span objects in development, which respectively represent a document, a single token, and a span. A container can contain other containers, for example, a document contains tokens and spans of text.

Token I

A Token object represents a word. Token objects are the building blocks of Doc and Span objects. The following properties of the Token class are available (see the documentation) :

```
- token.text: token text
- token.text_with_ws: token text with white spaces
- token.i: token index
- token.idx: position of the token's first character in the document
- token.doc: document containing the token
- token.sent: sentence containing the token
- token.is_sent_start: indicates if the token marks the start of a
  sentence
- token.ent_type: entity type of the token
```

span I

Span objects represent sentences or segments of text. Technically, a Span must be a contiguous sequence of tokens. We usually do not initialize Span objects, rather we slice a Doc object.

Trying to slice an invalid index will raise an IndexError. Most Python string indexing and slicing rules apply to Doc slicing too.

It's perfectly fine if you don't remember all the features going forward. If you don't remember the name of a feature, you can always do dir(token) or dir(doc). Calling dir() will display all the features and methods available on the object.

```
1  doc = nlp("I know you went to the USA.")
2  doc[2:4]
3  "you went"
```

There is another way to create a Span object - we can do a character-level slice of a Doc object using doc.char_span.

spaCy Linguistic Features I

- **Recap : Sentence Detection**

Sentence detection is the process that locates the start and end of sentences in a given text.

This allows us to break down the text into linguistically meaningful units.

We use these units when analyzing your text to perform tasks such as part-of-speech tagging (POS) and entity extraction (NER).

In spaCy, the `sents` property is used to extract sentences.

- **Recap : Text Tokenization**

spaCy Linguistic Features II

Tokenization is the next step after sentence detection. It allows you to identify the basic units of your text. These basic units are called tokens. Tokenization is useful because it breaks down text into meaningful units. These units are used for further analysis, such as part-of-speech tagging. In spaCy, you can print tokens by iterating over the Doc object.

▶ **Recap : Lemmatization**

Lemmatization is the process of reducing inflected forms of a word while ensuring that the reduced form belongs to the language. This reduced form or root word is called a lemma. For example, organizes, organized, and organizing are all forms of organize. Here,

spaCy Linguistic Features III

organize is the lemma. Inflection of a word allows you to express different grammatical categories like tense (organized vs organize), number (trains vs train), etc. Lemmatization is necessary because it helps you reduce the inflected forms of a word so they can be analyzed as a single item. It can also help you normalize the text. spaCy has the `lemma_` attribute on the Token class. This attribute has the lemmatized form of a token.

POS I

The acronym POS tagging stands for part-of-speech tagging. A part of speech is a syntactic category into which each word falls into a category according to its function in a sentence. For example, English has nine main categories : verb, noun, pronoun, determiner, adjective, adverb, preposition, conjunction, and interjection. We can describe the functions of each category as follows :

- **Verb** - **Verb** : Expresses an action or a state of being
- **Noun** - **Noun** : Identifies a person, place, or thing, or names one of these items (a proper noun)

POS II

- **Pronoun** - **Pronoun** : Can replace a noun or a noun phrase
- **Determiner** - **Determiner** : Is placed in front of a noun to express a quantity or specify what the noun refers to - briefly, a noun introducer
- **Adjective** - **Adjective** : Modifies a noun or a pronoun
- **Adverb** - **Adverb** : Modifies a verb, an adjective, or another adverb
- **Preposition** - **Preposition** : Connects a noun/pronoun to other parts of the sentence

POS III

- **Conjunction - Conjunction** : Glues words, clauses, and sentences together
- **Interjection - Interjection** : Expresses emotion in a sudden and exclamatory manner

This basic set of categories, without any language-specific morphological or syntactic feature, are called universal tags. spaCy captures the universal tags via the pos_feature feature and describes them with examples.

It's time for some code now. Again, spaCy offers universal POS tags via the token.pos (int) and token.pos_ (unicode) features. Fine-grained POS tags are available via the

POS IV

token.tag (int) and token.tag_ (unicode) features. Let's learn more about the tags you'll encounter the most, through some examples. The following example includes examples of noun, proper noun, pronoun, and verb tags :

```
import spacy
nlp = spacy.load("en_core_web_md")
doc = nlp("Alicia and me went to the school by bus.")
for token in doc:
print(token.text, token.pos_, token.tag_,)
spacy.explain(token.pos_), spacy.explain(token.tag_)
...
('Alicia', 'PROPN', 'NNP', 'proper noun', 'noun, proper singular')
('and', 'CCONJ', 'CC', 'coordinating  conjunction', 'conjunction,
  coordinating')
('me', 'PRON', 'PRP', 'pronoun', 'pronoun,  personal')
('went', 'VERB', 'VBD', 'verb', 'verb, past   tense')
('to', 'ADP', 'IN', 'adposition',   'conjunction, subordinating or
```

```
    preposition')
13  ('school', 'NOUN', 'NN', 'noun', 'noun, singular or mass')
14  ('with', 'ADP', 'IN', 'adposition', 'conjunction, subordinating or
    preposition')
15  ('bus', 'NOUN', 'NN', 'noun', 'noun, singular or mass')
16  ('.', 'PUNCT', '.', 'punctuation', 'punctuation mark, sentence
    closer')
```

NER I

Named Entity Recognition (NER) is the process that locates named entities in unstructured text and then classifies them into predefined categories such as person names, organizations, locations, monetary values, percentages, time expressions, etc. You can use NER to learn more about the meaning of your text. For example, you could use it to populate tags for a set of documents to improve keyword search. You could also use it to categorize customer support tickets into relevant categories. spaCy has the `ents` property on Doc objects. You can use it to extract named entities.

NER II

TYPE	DESCRIPTION
PER	Named person or family.
LOC	Name of politically or geographically defined location (cities, provinces, countries, international regions, bodies of water, mountains).
ORG	Named corporate, governmental, or other organizational entity.
MISC	Miscellaneous entities, e.g. events, nationalities, products or works or art.

Figure – Most common entity types

```
1
2  import spacy
3
4  # Load the spaCy language model (you can change "en_core_web_sm" to
     the model you are using)
5  nlp = spacy.load("en_core_web_sm")
6
7  # Example
8  piano_class_text = ('The Great Piano Academy is located'
9  ' in Mayfair or in the City of London and has'
10 ' world-class piano instructors.')
11 piano_class_doc = nlp(piano_class_text)
12 for ent in piano_class_doc.ents:
```

```
print(ent.text, ent.start_char, ent.end_char,
ent.label_, spacy.explain(ent.label_))
```

Example

```
piano_class_text = ('The Great Piano Academy is located'
' in Mayfair or in the City of London and has'
' world-class piano instructors.')
piano_class_doc = nlp(piano_class_text)
for ent in piano_class_doc.ents:
print(ent.text, ent.start_char, ent.end_char,
ent.label_, spacy.explain(ent.label_))

[Out] : Great Piano Academy 0 19 ORG Companies, agencies,
 institutions, etc.
```

```
Mayfair 35 42 GPE Countries, cities, states
the City of London 46 64 GPE Countries, cities, states

#Here is what the different parts of the code mean:
#ent.text gives the Unicode text representation of the entity.
#ent.start\_char indicates the character offset for the start of the
   entity.
#ent.end\_char indicates the character offset for the end of the
  entity.
#ent.label\_ gives the label of the entity.
#spacy.explain(ent.label\_) gives an explanation of the entity label
  .
```